More practice - Strings and parsing

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	2	2
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Strings and parsing | 5 questions | tap a title to jump to it

E	1. Implement string formatting and request tracking tasks	PracHub	2026-02-11
M	2. Check Whether Digits Are Monotone Increasing	PracHub	2026-07-04
M	3. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)	1Point3Acres	2026-02-01
H	4. Find Dictionary Matches for a Jammed String	PracHub	2026-04-09
H	5. Convert Valid Snake Case to Lower Camel Case	PracHub	2026-04-16

STUDY PATH: question 136 of 167 (more practice)

Priority score 0.96 | Strings and parsing | asked 251 days ago

EASY | PracHub | Technical Screen | 1 of 5

Implement string formatting and request tracking tasks

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 159/228

ABOUT THIS QUESTION

Topic: More practice - Strings and parsing

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

LeetCode match: LC 246 Strobogrammatic Number (+ formatting and tracking subtasks) (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-05-21)

Original: https://prachub.com/interview-questions/implement-string-formatting-and-request-tracking-tasks

QUESTION

Overview: This multi-part question evaluates string manipulation, numeric formatting, character-mapping validation under 180° rotation, and event-tracking/order management competencies, assessing parsing, edge-case handling, mapping validation, and design of data structures for ordered output.

APPROACH

Multi-part: numeric/string formatting, validating a number that reads the same after a 180 degree rotation (digit map 0-0, 1-1, 6-9, 8-8, 9-6, two-pointer check), and an ordered request tracker (dict + insertion order). Solve each part independently.

INTERVIEW-DAY PLAN

CLARIFY FIRST

This is four small tasks, so confirm each contract. For the rotation task ask what the rotation map contains and whether leading zeros are allowed. For the tracker ask what 'print in order' means when a later request can still have an earlier end time.

BRUTE FORCE TO SAY OUT LOUD

For each part write the direct version first: split and join, loop with commas, check both readings, and print immediately.

OPTIMAL IDEA

Snake to camel: split on underscores, drop empty parts, capitalize all but the first. Thousands separators: format the absolute value in groups of three from the right and restore the sign. OCR check: validate as-is and after reversing and mapping each character, reject unmapped characters. Tracker: a map for the first timestamp, a min-heap of completed (end, id) and a drain with a watermark.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

snake: leading, trailing and repeated underscores. Commas: 0, negative, 64-bit minimum. OCR: unmapped letter, value 201, value 0. Tracker: end before begin, equal end times.

LIKELY FOLLOW-UPS

Locale-aware formatting, rotation maps given at runtime, and bounded memory in the tracker.

PITFALLS

Negating the minimum 64-bit integer. Printing a request before it is safe to order.

HOW TO THINK ABOUT IT

Four independent tasks, so solve and test each separately and say the contract for each before coding: what an empty or negative input returns, what 'valid' means for the rotated number, and for the tracker when it is safe to print in order. For the tracker, completion is the second event of a pair, and order printing is a heap that is drained only below a watermark.

Basis: Four small tasks: snake_case to camelCase, thousands separators, a rotated-OCR check (LeetCode 246 / 1056 flavor), and a request begin/end tracker that prints in end-time order.

BRUTE FORCE · Task A by walking characters

BRUTE FORCE · PSEUDO-CODE

```
def to_camel(s):
    result, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            result += ch.upper() if up else ch
            up = False
    return result
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear and fine; the split-based version is shorter. The only care is leading and repeated underscores.

BRUTE FORCE · Task B with the language's format specifier

BRUTE FORCE · PSEUDO-CODE

```
def format_number(n):
    return format(n, ",")  # built-in separator
```

TIME	SPACE
O(d)	O(d)

WHY IT IS SLOW

BRUTE FORCE · Task B with the language's format specifier

Correct and O(d), but it hides the algorithm; interviews usually ask for the grouping by hand: peel three digits at a time from the right.

BRUTE FORCE · Task C by checking all three readings with a straightforward helper

BRUTE FORCE · PSEUDO-CODE

```
def is_valid_rotation(text):
    if in_range(text):  # a number 1..200
        return True
    if not all(ch in ROTATE for ch in text):
        return False
    turned = [ROTATE[c] for c in reversed(text)]
    return in_range("".join(turned))
```

TIME	SPACE
O(L)	O(L)

WHY IT IS SLOW

It is already linear; there is little to speed up. The risk is correctness: unmapped characters make the rotation invalid, and leading zeros are not valid numbers.

BRUTE FORCE · Task D by storing all events and sorting when asked

BRUTE FORCE · PSEUDO-CODE

```
begin, end = {}, {}
def on_begin(rid, t):
    begin[rid] = t
def on_end(rid, t):
    end[rid] = t
def print_finished():
    done = [r for r in end if r in begin]
    done.sort(key=lambda r: (end[r], r))
    for r in done:
        print(r, begin[r], end[r])
        del begin[r], end[r]
```

TIME	SPACE
O(n log n) per print	O(n)

WHY IT IS SLOW

Each print sorts everything pending, O(n log n); a min-heap of completed requests prints in order with O(log n) per event, releasing only entries below a safe watermark.

OPTIMAL SOLUTION · Task A: snake_case to camelCase

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task B: thousands separators

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task C: valid as-is or after a 180 degree rotation

TIME	SPACE
O(L)	O(L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task D: request tracker that prints completed requests in end-time order

TIME	SPACE
O(log n) per event	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not print as soon as a request completes in the follow-up?

Because another request that completes later can still have an earlier end time. An ordered printer needs a contract for when it is safe to emit: either a watermark (everything with end <= W is final because ends arrive within some known lateness), or a flush at the end of input. State that contract, then the heap pops only below the watermark.

FOLLOW-UP 2 How do you handle bad input (duplicate begin, end before begin, never-completed requests)?

Duplicate begins overwrite or are rejected by policy; an end timestamp earlier than its begin is flagged as invalid rather than printed. Requests that stay in the pending map are the leak to watch: evict by age with a timer or a second heap keyed on first-seen time, and report them as incomplete.

FOLLOW-UP 3 Rotation map provided at runtime with extra pairs.

Nothing changes: the map is a parameter. Validate that it is consistent (rot of rot is the original) if you control it, and treat characters that are not in it as invalid.

FOLLOW-UP 4 The tracker must bound its memory.

Evict requests whose begin or end has been waiting longer than a retention window (a second heap keyed by first-seen time) and report them as incomplete. **Cost:** O(log n) per event for the eviction heap; memory O(requests inside the retention window).

FOLLOW-UP 5 Another locale: a different separator or grouping (Indian style 12,34,567).

Parameterise the separator and the group size: the lowest group may differ from the others (3 digits, then groups of 2 in the Indian system). The loop peels a group from the right each time.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement string formatting and request
# tracking tasks
#
# This multi-part question evaluates string manipulation,
# numeric formatting, character-mapping validation under
# 180° rotation, and event-tracking/order management
# competencies, assessing parsing, edge-case handling,
# mapping validation, and design of data structures for
# ordered output.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not print as soon as a request completes in the
# follow-up?
# 2. How do you handle bad input (duplicate begin, end
# before begin, never-completed requests)?
# 3. Rotation map provided at runtime with extra pairs.
# 4. The tracker must bound its memory.
# 5. Another locale: a different separator or grouping
# (Indian style 12,34,567).
#
# INTERVIEW NOTES
# Four independent tasks, so solve and test each
# separately and say the contract for each before coding:
# what an empty or negative input returns, what 'valid'
# means for the rotated number, and for the tracker when
# it is safe to print in order. For the tracker,
# completion is the second event of a pair, and order
# printing is a heap that is drained only below a
# watermark.
#
# BRUTE FORCE (say it first): Task A by walking characters
# Walk the identifier, set a flag after each underscore,
# and uppercase the next letter. It ignores leading
# underscores by construction when the flag is applied to
# the first output letter, so handle that case explicitly.
# Time O(n). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Task A by walking characters
# | time O(n)
def snake_to_camel_walk(s):
    out, up, started = [], False, False
    for ch in s:
        if ch == "_":
            up = started
        else:
            out.append(ch.upper() if up else ch)
            up, started = False, True
    return "".join(out)


# HELPERS for the base question
ROTATE = {"0": "0", "1": "1", "6": "9",
          "8": "8", "9": "6"}


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def snake_to_camel(s):
    # foo_bar -> fooBar (empty parts are ignored)
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    rest = [w.capitalize() for w in words[1:]]
    return words[0] + "".join(rest)


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
# FOLLOW-UP 5: Another locale: a different separator or
# grouping (Indian style 12,34,567).
# Answer: Parameterise the separator and the group size:
# the lowest group may differ from the others (3 digits,
# then groups of 2 in the Indian system). The loop peels a
# group from the right each time.
def format_number(n, sep=",", sizes=(3,)):
    # FOLLOW-UP 5: sep= and sizes=(3, 2) for other locales
    # group the digits from the right: 1234567 ->
    # 1,234,567.
    # sizes = (3, 2) is the Indian style 12,34,567: the
    # first
    # group has 3 digits, every later one has 2.
    sign = "-" if n < 0 else ""
    digits = str(abs(n))
    parts, i = [], 0
    while len(digits) > sizes[min(i, len(sizes) - 1)]:
        size = sizes[min(i, len(sizes) - 1)]
        # peel one group off the right end
        parts.append(digits[-size:])
        digits = digits[:-size]
        i += 1
    parts.append(digits)
    return sign + sep.join(reversed(parts))


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def is_rotation_valid(s, mapping=ROTATE):
    # does the number read the same upside down (180
    # degrees)?
    # a character not in the map makes it invalid
    if any(ch not in mapping for ch in s):
        return False
    # rotate every digit and read the result backwards
    turned = "".join(mapping[ch] for ch in reversed(s))
    return turned == s
```

```python
# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
class RequestTracker:
    """begin(id, t) and end(id, t). Finished requests are
    printed in the order they BEGAN: one is printed only
    when everything that began before it is done."""

    def __init__(self):
        self.order = deque()
        self.begin_at, self.end_at = {}, {}

    def begin(self, rid, t):
```

```python
        if rid in self.begin_at:
            raise ValueError("duplicate begin")
        self.begin_at[rid] = t
        self.order.append(rid)

    def end(self, rid, t):
        if rid not in self.begin_at:
            raise ValueError("end without begin")
        if t < self.begin_at[rid]:
            raise ValueError("ends before it begins")
        self.end_at[rid] = t
        # print from the front while the oldest are
        # finished
        out = []
        while self.order and self.order[0] in self.end_at:
            first = self.order.popleft()
            out.append((first, self.begin_at[first],
                        self.end_at[first]))
        return out


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("__a__b_") == "aB"
    assert snake_to_camel("") == ""
    assert format_number(1234567) == "1,234,567"
    assert format_number(-1000) == "-1,000"
    assert format_number(999) == "999"
    assert format_number(0) == "0"
    indian = format_number(1234567, sizes=(3, 2))
    assert indian == "12,34,567"
    assert format_number(1234, ".") == "1.234"
    assert is_rotation_valid("69")
    assert is_rotation_valid("818")
    assert not is_rotation_valid("962")
    assert not is_rotation_valid("12")
    assert is_rotation_valid("") is True
    t = RequestTracker()
    t.begin("a", 1)
    t.begin("b", 2)
    # b finished first, but a has not: nothing can be
    # printed
    assert t.end("b", 5) == []
    # a finishes: both are printed, in begin order
    assert t.end("a", 7) == [("a", 1, 7), ("b", 2, 5)]
```

```python
    again = lambda: t.begin("a", 9)
    orphan = lambda: t.end("zz", 1)
    for call in (again, orphan):
        try:
            call()
            raise AssertionError("should fail")
        except ValueError:
            pass
```

FULL QUESTION TEXT FROM THE SITE

You are given several small coding tasks. Implement each as described.

Task A — Convert snake_case to camelCase

Write a function that converts an identifier from snake_case to camelCase.

Input: a string s containing lowercase letters a-z and underscores _ .
Output: the camelCase version.
Rules/edge cases:
Consecutive underscores should be treated as a single word separator (ignore empty parts).
If s starts/ends with underscores, ignore those underscores.
The first word remains lowercase; each subsequent word is capitalized.

Example:

"hello_world" -> "helloWorld"
"__my__var__name_" -> "myVarName"
Task B — Format an integer with thousands separators

Write a function that formats a (possibly negative) integer with commas as thousands separators.

Input: an integer n (may be large; assume it fits in 64-bit signed integer).
Output: a string representation using commas every 3 digits from the right.

Examples:

123 -> "123"
1234 -> "1,234"
-12345678 -> "-12,345,678"
Task C — Validate an OCR string under 180° rotation

An OCR system outputs a string t that may contain digits and a few letters. You need to decide whether it can represent an integer between 1 and 200 inclusive under either normal reading or after a 180-degree rotation.

Define a rotation mapping rot(c) for characters; rotation is valid only if every character in the string has a mapping. Rotating a whole string means:

reverse the string, and
replace each character c with rot(c) .

Return true if either:

t as-is is a valid base-10 integer in [1, 200] , OR
rotate180(t) is a valid base-10 integer in [1, 200] .

Assume the rotation map is provided as a dictionary (examples from the interview note included):

'6' <-> '9'
plus any additional pairs given at runtime (e.g., 'h' -> '4' , 'L' -> '7'). Characters not in the map are invalid for rotation.

Examples (assuming map includes 6<->9, 0->0, 1->1, 8->8):

FULL QUESTION TEXT FROM THE SITE

"69" rotates to "69" and is valid (69 in range) -> true
"201" is out of range, rotate may or may not help -> depends on mapping -> compute
Task D — Track request begin/end times and print in a required order

You are implementing a Server class that receives events for requests. Each request has a unique requestId.

You must implement two methods:

begin(requestId, timestamp)
end(requestId, timestamp)
Part 1

Whenever both timestamps for a request are known, print/log a line:

requestId begin=<beginTs> end=<endTs>

Assumptions:

Events may arrive out of order (an end can arrive before begin). There may be multiple in-flight requests.
Follow-up

Instead of printing immediately when a pair is complete, the server must always print completed requests in increasing order of end time (earliest end printed first). If multiple have the same end time, break ties by requestId.

Design the data structures and logic to support this ordering efficiently.

STUDY PATH: question 137 of 167 (more practice)

Priority score 2.0 | Strings and parsing | asked 108 days ago

MEDIUM | PracHub | Technical Screen | 2 of 5

Check Whether Digits Are Monotone Increasing

POSTED 2026-07-04 | 108 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 193/228**

ABOUT THIS QUESTION

Topic: More practice - Strings and parsing

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy digit scan) (inferred from overview)

Posted on PracHub: 2026-07-04 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/check-whether-digits-are-monotone-increasing

QUESTION

Overview: Check whether a non-negative integer's decimal digits are non-decreasing from left to right. Cover zero, single-digit values, repeated digits, and an O(d) implementation using either arithmetic digit extraction or string traversal.

APPROACH

Compare each digit with the previous one using % 10 and // 10 from the right (or string scan); handle 0 and single digits. O(d).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether equal neighbors are allowed (non-decreasing), whether the input can be negative, and whether to use the integer or string form.

BRUTE FORCE TO SAY OUT LOUD

Convert to a string and compare each pair. Fine and O(d).

OPTIMAL IDEA

Without strings: peel digits from the right with % 10 and // 10, keeping the previous digit; fail if the current digit is greater than the previous one seen to its right. O(d), O(1) space.

DRY-RUN THESE CASES

0, a single digit, 1234, 1224, 1243, 100.

LIKELY FOLLOW-UPS

Strictly increasing, or find the next monotone number.

PITFALLS

Comparing in the wrong direction when scanning right to left.

HOW TO THINK ABOUT IT

Compare neighbouring digits. You can read the digits from the right with % 10 and // 10, remembering the previous (right-hand) digit: the number is monotone if no digit is larger than the one to its right. Say first whether equal neighbours are allowed.

Basis: The check behind LeetCode 738 (non-decreasing digits).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_monotone(x):
    s = str(x)
    return s == "".join(sorted(s))
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorting costs O(d log d) and copies the digits. Comparing each digit with its neighbour while peeling digits with % 10 is O(d) with O(1) space.

OPTIMAL SOLUTION · Peel digits from the right and compare with the previous one

TIME	SPACE
O(d)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Strictly increasing digits.

Use >= instead of > when comparing with the previous digit, and start prev at 10 so a single digit passes.

FOLLOW-UP 2 Negative numbers or numbers in another base.

Decide what monotone means for the sign (usually apply it to the absolute value). For base b replace 10 with b in % and //.

FOLLOW-UP 3 Find the next monotone number after n.

That is the left-to-right fix from the previous question: find the first descent and copy that digit across the rest.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check Whether Digits Are Monotone
# Increasing
#
```

```
# Check whether a non-negative integer's decimal digits
# are non-decreasing from left to right. Cover zero,
# single-digit values, repeated digits, and an O(d)
# implementation using either arithmetic digit extraction
# or string traversal.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Strictly increasing digits.
# 2. Negative numbers or numbers in another base.
# 3. Find the next monotone number after n.
#
# INTERVIEW NOTES
# Compare neighbouring digits. You can read the digits
# from the right with % 10 and // 10, remembering the
# previous (right-hand) digit: the number is monotone if
# no digit is larger than the one to its right. Say first
# whether equal neighbours are allowed.
#
# BRUTE FORCE (say it first): String comparison with its
# sorted version
# Convert to a string and compare it with the sorted
# string: a number has non-decreasing digits exactly when
# its digits are already in sorted order. Clear, with an
# unneeded sort. Time O(d log d). Space O(d).


# BRUTE FORCE (say it first): String comparison with its
# sorted version | time O(d log d)
def digits_monotone_sorted(n):
    s = str(n)
    return s == "".join(sorted(s))


# BASE QUESTION (optimal): Peel digits from the right and
# compare with the previous one
# Idea: Take the last digit with n % 10, then the next
# one, and so on. Because we move right to left, the digit
# just read must not exceed the one read before it. A
# single digit and zero are monotone.
def is_monotone_digits(n):
    # non-decreasing digits, read from the right with n %
    # 10
    prev = 9
    while n:
        d = n % 10
        # a digit bigger than the one to its right: a
        # descent
        if d > prev:
            return False
        prev = d
        n //= 10
    return True


# SAME TEST on the text of the number
def is_monotone_string(n):
    # the same test on the text of the number
    s = str(n)
    return all(a <= b for a, b in zip(s, s[1:]))


# FOLLOW-UP 1: Strictly increasing digits.
# Answer: Use >= instead of > when comparing with the
# previous digit, and start prev at 10 so a single digit
# passes.
def is_strictly_monotone(n):
    # strictly increasing: start prev at 10, compare with
    # >=
    prev = 10
    while n:
        d = n % 10
        if d >= prev:
            return False
        prev = d
        n //= 10
    return True


# FOLLOW-UP 2: Negative numbers or numbers in another
# base.
# Answer: Decide what monotone means for the sign (usually
# apply it to the absolute value). For base b replace 10
# with b in % and //.
def is_monotone_base(n, base=10):
    # any base: use the base for % and //
    prev = base - 1
    while n:
        d = n % base
        if d > prev:
            return False
        prev = d
        n //= base
    return True


# FOLLOW-UP 3: Find the next monotone number after n.
# Answer: That is the left-to-right fix from the previous
# question: find the first descent and copy that digit
# across the rest.
def next_monotone(n):
    # smallest monotone number AFTER n: fix the first
    # descent
    d = list(str(n + 1))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            d[i + 1:] = d[i] * (len(d) - i - 1)
            break
    return int("".join(d))


if __name__ == "__main__":
    assert is_monotone_digits(0) and is_monotone_digits(7)
    assert is_monotone_digits(1223)
    assert is_monotone_digits(999)
    assert not is_monotone_digits(332)
    assert not is_monotone_digits(10)
    assert is_strictly_monotone(1234)
    assert not is_strictly_monotone(1224)
    assert is_strictly_monotone(5)
    # binary 11 -> 3 is monotone; binary 10 -> 2 is not
    assert is_monotone_base(3, 2)
    assert not is_monotone_base(2, 2)
    assert next_monotone(332) == 333
    assert next_monotone(9) == 11
    for n in range(2000):
        by_text = is_monotone_string(n)
        assert is_monotone_digits(n) == by_text
```

FUNCTION AND TEST CASES

Signature

```
def has_monotone_increasing_digits(n: int) -> bool:
SAMPLE INPUT
0
```

Test cases

Input	Expected
0	True
7	True

STUDY PATH: question 138 of 167 (more practice)

Priority score 1.5 | Strings and parsing | asked 261 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 3 of 5

Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 202/228

ABOUT THIS QUESTION

Topic: More practice - Strings and parsing

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/document-line-wrap

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space
    would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo
    bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

APPROACH

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the width counts characters in a monospace font, how to treat explicit newlines, multiple spaces and leading spaces, what to do with a word longer than the width, and whether to return the number of lines or the lines themselves.

BRUTE FORCE TO SAY OUT LOUD

Build the text line by line by trying to add each word and measuring. That is already the greedy algorithm.

OPTIMAL IDEA

Tokenize into words and newline tokens. Keep the current line length; for each word, if it fits (with a space unless the line is empty) add it, otherwise start a new line. A newline token always ends the current line. Count lines (empty lines count). Greedy is expected; mention Knuth's minimum-raggedness DP only if asked. O(n).

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

The example giving 4. width = 1. Consecutive newlines. A word exactly the width. A word longer than the width.

LIKELY FOLLOW-UPS

Return the lines, justify the text, or minimize raggedness.

PITFALLS

Dropping empty lines. Adding a trailing space at the end of a line.

HOW TO THINK ABOUT IT

Greedy is optimal for the question 'fewest lines': put as many words on a line as fit. The decision per word is only 'fits with a space', 'fits as the first word on the line', or 'start a new line', so track the current line length, not the line text. Treat newline as a hard break and decide up front what empty lines and long words do.

Basis: Written; greedy word wrap (LeetCode 68, Text Justification, without the justification step). Assumed rules (confirm them): monospace width in characters; words are separated by whitespace; '\n' always ends a line and an empty line counts; a word longer than the width gets its own line.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wrap(text, width):
    count = 0
    for paragraph in text.split("\n"):
        line = ""
        for word in paragraph.split():
            joined = len(line) + 1 + len(word)
            if line and joined > width:
                count += 1  # finish the line
                line = word
            else:
                line = line + " " + word
                line = line.strip()
        count += 1  # last line
    return count
```

TIME	SPACE
O(total characters * words per line)	O(width)

WHY IT IS SLOW

Re-measuring a joined string for every word costs extra copying. Keeping a running length makes the whole thing O(total characters).

OPTIMAL SOLUTION · Greedy with a running line length

TIME	SPACE
O(n)	O(1) extra beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the lines, not only the count.

Keep the same loop but append the finished line to a list (the brute force above already does it). Build each line from a list of words and join once, so each word is copied once.

FOLLOW-UP 2 A word longer than the width must be broken.

Before placing a word, while it is longer than the width flush the current line and emit width-sized chunks of the word, then continue with the remainder as a normal word.

FOLLOW-UP 3 Minimise raggedness instead of lines.

That is Knuth's word-wrap DP: dp[i] is the best cost of laying out words i.. with cost (spaces left)^2 on each line except the last; O(n * words per line). Greedy is not optimal for this objective.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Google Doc Line Wrap (Word Wrap with
# Whitespace and Newlines)
#
# Examples: text = "Hello world\nFoo bar baz qux" width =
# 10 Line 1: "Hello" (5 chars; "world" + space would be 11
# -> break) Line 2: "world" Line 3: "Foo bar" (7 chars;
# "baz" -> "Foo bar baz" = 11 -> break) Line 4: "baz qux"
# Output: 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the lines, not only the count.
# 2. A word longer than the width must be broken.
# 3. Minimise raggedness instead of lines.
#
# INTERVIEW NOTES
# Greedy is optimal for the question 'fewest lines': put
# as many words on a line as fit. The decision per word is
# only 'fits with a space', 'fits as the first word on the
# line', or 'start a new line', so track the current line
# length, not the line text. Treat newline as a hard break
# and decide up front what empty lines and long words do.
#
# BRUTE FORCE (say it first): Build each line as a string
# and measure it
# Try appending each word to the current line string and
# check its length, starting a new line when it exceeds
# the width. Correct, but joining strings repeatedly costs
# more than the length counter in the optimal version.
# Time O(total characters * words per line). Space
# O(width).


# BRUTE FORCE (say it first): Build each line as a string
# and measure it | time O(total characters * words per
# line)
def wrap_lines_brute(text, width):
    lines = []
    for paragraph in text.split("\n"):
        words = paragraph.split()
        cur = ""
        for w in words:
            trial = (cur + " " + w) if cur else w
            if len(trial) <= width or not cur:
                cur = trial
            else:
                lines.append(cur)
                cur = w
        lines.append(cur)
    return lines


# HELPERS for the base question
# FOLLOW-UP 1: Return the lines, not only the count.
# Answer: Keep the same loop but append the finished line
# to a list (the brute force above already does it). Build
# each line from a list of words and join once, so each
# word is copied once.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def wrap_lines(text, width, break_long=False):
    lines = []
    # a newline always starts a new paragraph (and a new
    # line)
    for paragraph in text.split("\n"):
        cur = ""
        words = paragraph.split()
        if break_long:
            pieces = [split_long(w, width) for w in words]
            words = [p for chunk in pieces for p in chunk]
        for w in words:
            if not cur:
                cur = w
            elif len(cur) + 1 + len(w) <= width:
                # the word and one space still fit
                cur += " " + w
            else:
                lines.append(cur)
                cur = w
        # the last (or only, or empty) line of the
        # paragraph
        lines.append(cur)
    return lines


# FOLLOW-UP 2: A word longer than the width must be
# broken.
# Answer: Before placing a word, while it is longer than
# the width flush the current line and emit width-sized
# chunks of the word, then continue with the remainder as
# a normal word.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def split_long(word, width):
    # a word longer than the line is cut into pieces
    starts = range(0, len(word), width)
    return [word[i:i + width] for i in starts]


# BASE QUESTION (optimal): Greedy with a running line
# length
# Idea: Split on newlines first (each paragraph ends its
# line), then for each word compare cur_len + 1 +
# len(word) with the width. Only the length is tracked,
# never the joined line.
def wrap_count(text, width):
    return len(wrap_lines(text, width))


# FOLLOW-UP 3: Minimise raggedness instead of lines.
# Answer: That is Knuth's word-wrap DP: dp[i] is the best
# cost of laying out words i.. with cost (spaces left)^2
# on each line except the last; O(n * words per line).
# Greedy is not optimal for this objective.
```

```python
def min_raggedness(words, width):
    # Knuth: minimise the squared spare space of every
    # line
    # except the last; greedy is NOT optimal for this goal
    n = len(words)
    INF = float("inf")
    # cost[i]: best total cost for words[i:]
    cost, nxt = [INF] * (n + 1), [0] * (n + 1)
    cost[n] = 0
    for i in range(n - 1, -1, -1):
        length = -1
        for j in range(i, n):
            length += len(words[j]) + 1
            if length > width:
                break
            spare = width - length
            # the last line costs nothing
            extra = 0 if j == n - 1 else spare ** 2
            if extra + cost[j + 1] < cost[i]:
                cost[i] = extra + cost[j + 1]
                nxt[i] = j + 1
    lines, i = [], 0
```

```python
    while i < n:
        lines.append(" ".join(words[i:nxt[i]]))
        i = nxt[i]
    return cost[0], lines


if __name__ == "__main__":
    text = "Hello world\nFoo bar baz qux"
    assert wrap_lines(text, 10) == [
        "Hello", "world", "Foo bar", "baz qux"]
    assert wrap_count(text, 10) == 4
    # blank lines count as lines
    assert wrap_count("a\n\nb", 5) == 3
    # a long word is split when asked
    cut = wrap_lines("abcdefgh", 3, True)
    assert cut == ["abc", "def", "gh"]
    assert wrap_lines("abcdefgh", 3) == ["abcdefgh"]
    # the best layout can beat the greedy one
    cost, lines = min_raggedness(
        "aaa bb cc ddddd".split(), 6)
    assert lines == ["aaa", "bb cc", "ddddd"]
    assert cost == 9 + 1
```

NOTES FROM CANDIDATES

Single-pass tokenizer + accumulator: track curLen; on each token, decide "fits with leading space", "fits without space" (start of line), or "flush and start new line".
Watch out for \n right after another \n (empty line counts), \n at the end of input, leading whitespace, multiple consecutive spaces inside a line.
Don't try to handle font / kerning / hyphenation — the prompt assumes monospace width counted in characters.
Greedy packing is the expected answer (LC 68 style). Mention DP word-wrap (Knuth's algorithm) only if the interviewer pushes on 'minimum raggedness'; greedy is optimal-enough and what graders look for. Watch edge cases: a single word longer than W, an explicit newline forcing a break mid-pack, trailing whitespace handling.

PREPARATION

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

STUDY PATH: question 139 of 167 (more practice)

Priority score 1.52 | Strings and parsing | asked 194 days ago

HARD | PracHub | Technical Screen | 4 of 5

Find Dictionary Matches for a Jammed String

POSTED 2026-04-09 | 194 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 134/228

ABOUT THIS QUESTION

Topic: More practice - Strings and parsing

Optimal answer from: the interview site's own solution

LeetCode match: LC 809 Expressive Words (inferred from overview)

Posted on PracHub: 2026-04-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/find-dictionary-matches-for-a-jammed-string

QUESTION

Overview: This question evaluates string-processing and pattern-matching skills, specifically run-length grouping of consecutive characters and efficient dictionary lookup, within the Coding & Algorithms domain.

APPROACH

Run-length encode the query and every dictionary word; a word matches if run characters align and each run length is compatible. Pre-group dictionary by compressed signature in a hash map for fast lookup.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what jammed means exactly: a letter repeated more than it should be, and whether the repeat count must be at least the word's count or at least some threshold. Confirm case, empty strings and whether a match requires the same letter order and same number of runs.

BRUTE FORCE TO SAY OUT LOUD

For each dictionary word, compare it with the query run by run. O(W * L) per query.

OPTIMAL IDEA

Compress each word into runs of (letter, count). Use the letter sequence (without counts) as a hash key so only words with the same run letters are candidates, then compare the counts under the agreed rule. For repeated queries a trie over run letters keeps lookups proportional to the number of runs.

DRY-RUN THESE CASES

Query 'heeelllo' against 'hello' and 'helo'. A query with fewer runs than the word. A single-letter word. A word whose runs are longer than the query's.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Stretch only allowed for runs of three or more, return all matches in input order, handle a huge dictionary with a trie.

PITFALLS

Treating 'ee' to 'e' as a match without checking the rule. Comparing total length instead of per-run counts.

HOW TO THINK ABOUT IT

'Same letters, stretched' means compare run-length encodings, not characters. The letter order of the runs has to match exactly, so the sequence of distinct run letters is a hash key that cuts the dictionary down before any counts are compared.

Basis: LeetCode 809, Expressive Words. Assumed rule (confirm it): a query run may be longer than the word's run only when the query run has length at least 3; otherwise the lengths must be equal.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def matches(query, words):
    runs_q = runs(query)  # [(letter, count)]
    result = []
    for word in words:
        runs_w = runs(word)
        if compatible(runs_w, runs_q):
            result.append(word)
    return result
```

TIME	SPACE
O(W * L) per query	O(L)

WHY IT IS SLOW

Every query touches every word, O(W * L), even words whose letters cannot match. Grouping words by their run letters in a hash map checks only the right bucket.

OPTIMAL SOLUTION · Index the dictionary by its run letters

TIME	SPACE
O(L) per query plus the size of the matching bucket	O(total dictionary length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A different stretch rule: any run may be longer, no minimum.

Only the comparison changes. The index (grouping by run letters) is the same, so make the minimum stretch a parameter, as the code does with min_stretch.

FOLLOW-UP 2 Millions of dictionary words and many queries.

Keep the group index in memory and put a trie over the run-letter strings if the keys share long prefixes. For counts compare only the first words in a bucket that survive a cheap check such as total length. **Cost:** O(L) per query to build the run key plus the size of the bucket scanned; building the index is O(total dictionary length) once.

FOLLOW-UP 3 The jammed string may also be missing letters.

That becomes an edit-distance style problem on run encodings, usually solved with DP over (query run, word run). Ask for the exact rule before coding it.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find Dictionary Matches for a Jammed
# String
#
# This question evaluates string-processing and pattern-
# matching skills, specifically run-length grouping of
# consecutive characters and efficient dictionary lookup,
# within the Coding & Algorithms domain.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A different stretch rule: any run may be longer, no
# minimum.
# 2. Millions of dictionary words and many queries.
# 3. The jammed string may also be missing letters.
#
# INTERVIEW NOTES
# 'Same letters, stretched' means compare run-length
# encodings, not characters. The letter order of the runs
# has to match exactly, so the sequence of distinct run
# letters is a hash key that cuts the dictionary down
# before any counts are compared.
#
# BRUTE FORCE (say it first): Compare the query with every
# dictionary word
# Encode the query once and compare it with each word's
# runs. It is correct; the cost is that every query
# touches every word, even words whose letters cannot
# match. Time O(W * L) per query. Space O(L).


from itertools import groupby


# HELPERS for the brute force
def _runs(s):
    out = []
    for ch in s:
        if out and out[-1][0] == ch:
            out[-1][1] += 1
        else:
            out.append([ch, 1])
    return out


# BRUTE FORCE (say it first): Compare the query with every
# dictionary word | time O(W * L) per query
def _stretchy(query, word, min_stretch=3):
    a, b = _runs(query), _runs(word)
    if len(a) != len(b):
        return False
    return all(
        qa == wa
        and (
            qc == wc or (qc > wc and qc >= min_stretch)
        )
        for (qa, qc), (wa, wc) in zip(a, b)
    )


def jammed_matches_brute(words, query):
    return [w for w in words if _stretchy(query, w)]


# HELPERS for the base question
def runs(word):
    # "heello" -> [("h", 1), ("e", 2), ("l", 2), ("o", 1)]
    return [(ch, len(list(g))) for ch, g in groupby(word)]


# BASE QUESTION (optimal): Index the dictionary by its run
# letters
# Idea: Group words by the string of their distinct run
# letters ('hello' -> 'helo'). A query can only match
# words in its own group, so compare run counts only
# there. Build the index once and reuse it for many
# queries.
# FOLLOW-UP 1: A different stretch rule: any run may be
# longer, no minimum.
# Answer: Only the comparison changes. The index (grouping
# by run letters) is the same, so make the minimum stretch
# a parameter, as the code does with min_stretch.
class JammedIndex:
    """A query run may be longer than the word's run
    when it is at least min_stretch long."""

    def __init__(self, words):
        # words grouped by their run letters ("hello" ->
        # "helo")
        self.groups = {}
        for w in words:
            r = runs(w)
            letters = "".join(ch for ch, _ in r)
            self.groups.setdefault(letters, []).append(
                (w, [n for _, n in r]))

    # FOLLOW-UP 1: min_stretch=1: any run may be longer
    def matches(self, query, min_stretch=3):
        r = runs(query)
        letters = "".join(ch for ch, _ in r)
        counts = [n for _, n in r]
        found = []
        # only words with the same run letters can match
        group = self.groups.get(letters, [])
        for word, word_counts in group:
            if all(self.fits(q, w, min_stretch)
                   for q, w in zip(counts, word_counts)):
                found.append(word)
        return sorted(found)

    @staticmethod
    def fits(q, w, min_stretch):
        # the same length, or a stretch of at least
        # min_stretch
        return q == w or (q > w and q >= min_stretch)


if __name__ == "__main__":
    index = JammedIndex(["hello", "help", "hi", "hip"])
    # e stretched to 4 and o stretched to 3
```

```
    assert index.matches("heeeelloooo") == ["hello"]
    assert index.matches("hello") == ["hello"]
    # a stretch of only 2 is not enough
    assert index.matches("heello") == []
    assert index.matches("hiiii") == ["hi"]
    assert index.matches("zzz") == []
    # a smaller minimum accepts the short stretch
    short = index.matches("heello", min_stretch=2)
    assert short == ["hello"]
    # no minimum: any longer run is fine
    assert index.matches("hiip", min_stretch=1) == ["hip"]
    want = [("a", 2), ("b", 1), ("c", 3)]
    assert runs("aabccc") == want
```

FUNCTION AND TEST CASES

Signature

```
def solution(dictionary_words: list[str], jammed_string: s
tr) -> list[str]:
```

Sample call

```
solution(['help', 'hello', 'shell', 'hellp'], 'hellp')
```

Test cases

Input	Expected
(['help', 'hello', 'shell', 'hellp'], 'hellp')	['help', 'hellp']
(['hello', 'halo', 'heeello'], 'hheeellllo')	['hello', 'heeello']

STUDY PATH: question 140 of 167 (more practice)

Priority score 1.5 | Strings and parsing | asked 187 days ago

HARD | PracHub | Technical Screen | 5 of 5

Convert Valid Snake Case to Lower Camel Case

POSTED 2026-04-16 | 187 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 184/228

ABOUT THIS QUESTION

Topic: More practice - Strings and parsing

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy string manipulation)

Posted on PracHub: 2026-04-16 (updated 2026-07-18)

Original: https://prachub.com/coding-questions/convert-valid-snake-case-to-lower-camel-case

QUESTION

Convert Valid Snake Case to

Lower Camel Case Given a valid snake-case identifier, convert it to lower camel case. The input consists of one or more lowercase ASCII words separated by single underscores. There is no leading underscore, trailing underscore, or pair of adjacent underscores. The output keeps the first word lowercase, removes each underscore, and capitalizes the first letter of every later word. Implement:

```
def solve(name: str) -> str: ...
```

Constraints

- `1 <= len(name) <= 100_000`
- **name** matches `[a-z]+(?:_[a-z]+)*`

Examples

```
"foo_bar" -> "fooBar" "already" -> "already" "http_api_name" -> "httpApiName"
```

APPROACH

Split on '_', keep the first word lowercase, capitalize the first letter of the rest, join. O(n). The Hard tag on PracHub looks like a labeling quirk.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether input can have digits, uppercase, leading or trailing underscores, or double underscores. The problem states it is valid, so say you will rely on that and mention what you would add if not.

BRUTE FORCE TO SAY OUT LOUD

Loop over characters and capitalize the character after each underscore. Linear and simple.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Split on '_', keep the first word as is, capitalize the first letter of each other word and join. O(n). The Hard label on PracHub looks like a labeling quirk; do not over-engineer it.

DRY-RUN THESE CASES

A single word, two words, many words, a one-letter word.

LIKELY FOLLOW-UPS

Convert back from camel to snake, or handle acronyms.

PITFALLS

Capitalizing the first word. Using replace on every underscore without capitalizing.

HOW TO THINK ABOUT IT

The statement says the input is valid (single underscores, no edge underscores), so rely on that and say what you would add if it were not. The pattern is 'first word unchanged, capitalize the first letter of each later word'.

Basis: Written. The PracHub Hard label looks like a labeling quirk; this is an easy string task.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def to_camel(s):
    out, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            out += ch.upper() if up else ch
            up = False
    return out
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Already O(n); it is just longer than needed. Splitting on '_' and capitalising each word after the first is the short form.

OPTIMAL SOLUTION · Split on underscores and capitalize

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input may have leading, trailing or repeated underscores, or mixed case.

Drop empty parts after the split, lowercase the first word, capitalize the rest. Say what you do with digits and non-letters, and keep the rule in one place.

FOLLOW-UP 2 Convert back from camel case to snake case.

Insert an underscore before each uppercase letter and lowercase it. Acronyms such as HTTPServer need a smarter rule (split between a lowercase-to-uppercase edge and between an acronym and the next word).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Convert Valid Snake Case to Lower Camel
# Case
#
# # Convert Valid Snake Case to Lower Camel Case Given a
# valid snake-case identifier, convert it to lower camel
# case. The input consists of one or more lowercase ASCII
# words separated by single underscores. There is no
# leading underscore, trailing underscore, or pair of
# adjacent underscores. The output keeps the first word
# lowercase, removes each underscore, and capitalizes the
# first letter of every later word. Implement: python def
# solve(name: str) -> str: ... ## Constraints - 1 <=
# len(name) <= 100_000 - name matches [a-z]+(?:_[a-z]+)*
# ## Examples text "foo_bar" -> "fooBar" "already" ->
# "already" "http_api_name" -> "httpApiName"
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input may have leading, trailing or repeated
# underscores, or mixed case.
# 2. Convert back from camel case to snake case.
#
# INTERVIEW NOTES
# The statement says the input is valid (single
# underscores, no edge underscores), so rely on that and
# say what you would add if it were not. The pattern is
# 'first word unchanged, capitalize the first letter of
# each later word'.
#
# BRUTE FORCE (say it first): Walk the characters and
# capitalize after each underscore
# Keep a flag that is set after an underscore; when set,
# append the uppercase of the next letter. Linear and
# fine, just longer than the split-based answer. Time
# O(n). Space O(n).


import re


# BRUTE FORCE (say it first): Walk the characters and
# capitalize after each underscore | time O(n)
def snake_to_camel_brute(s):
    out, up = [], False
    for ch in s:
        if ch == "_":
```

```
            up = True
        else:
            out.append(ch.upper() if up else ch)
            up = False
    return "".join(out)


# BASE QUESTION (optimal): Split on underscores and
# capitalize
# Idea: Split on '_', keep the first word, and capitalize
# the first letter of each other word with a join. For
# valid input there are no empty parts.
def snake_to_camel(s):
    # foo_bar_baz -> fooBarBaz
    first, *rest = s.split("_")
    caps = [w[:1].upper() + w[1:] for w in rest]
    return first + "".join(caps)


# FOLLOW-UP 1: The input may have leading, trailing or
# repeated underscores, or mixed case.
# Answer: Drop empty parts after the split, lowercase the
# first word, capitalize the rest. Say what you do with
# digits and non-letters, and keep the rule in one place.
def snake_to_camel_tolerant(s):
    # leading, trailing or repeated underscores and mixed
    # case: drop empty parts, lowercase the first word
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    first = words[0].lower()
    rest = [w.capitalize() for w in words[1:]]
    return first + "".join(rest)


# FOLLOW-UP 2: Convert back from camel case to snake case.
# Answer: Insert an underscore before each uppercase
# letter and lowercase it. Acronyms such as HTTPServer
# need a smarter rule (split between a lowercase-to-
# uppercase edge and between an acronym and the next
# word).
def camel_to_snake(s):
    # fooBarBaz -> foo_bar_baz, and HTTPServer ->
    # http_server
    # 1) split an acronym from the next word: HTTPServer
```

```
    # -> HTTP_Server
    s = re.sub(r"([A-Z]+)([A-Z][a-z])", r"\1_\2", s)
    # 2) split a lower-case letter or digit from an upper-
    # case one
    s = re.sub(r"([a-z0-9])([A-Z])", r"\1_\2", s)
    return s.lower()


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    got = snake_to_camel("get_http_response")
    assert got == "getHttpResponse"
    # messy input
    messy = snake_to_camel_tolerant("_Foo__BAR_")
    assert messy == "fooBar"
    assert snake_to_camel_tolerant("___") == ""
    # and back again
    assert camel_to_snake("fooBarBaz") == "foo_bar_baz"
    assert camel_to_snake("HTTPServer") == "http_server"
    assert camel_to_snake("x") == "x"
    camel = snake_to_camel("my_var_2")
    assert camel_to_snake(camel) == "my_var2"
```

FUNCTION AND TEST CASES

Signature

```
def solve(name) -> str:
```

Sample call

```
solve('a')
```

Test cases

Input	Expected
`('a',)`	`'a'`
`('already',)`	`'already'`

ALSO REPORTED · 2 more

Define the scope and requirements for a snake_case to camelCase converter function. | Exponent

https://www.aced.io/questions/6328/define-scope-snake-case-camel-case-converter

Snake Case to Camel Case with Repository-Wide Refactor | 1Point3Acres | tech-screen | asked 2026-07-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

Preparation: Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.